

UNIVERSITY OF OSLO

A Self-Adaptive Digital Twin Architecture for Dynamic Resource Management

Riccardo Sieve, Paul Kobialka, Andrea
Pferscher, Nelly Bencomo, Lizeth Silvia
Tapia Tarifa, Buster Salomon Rasmussen,
Einar Broch Johnsen

SEAMS 2026

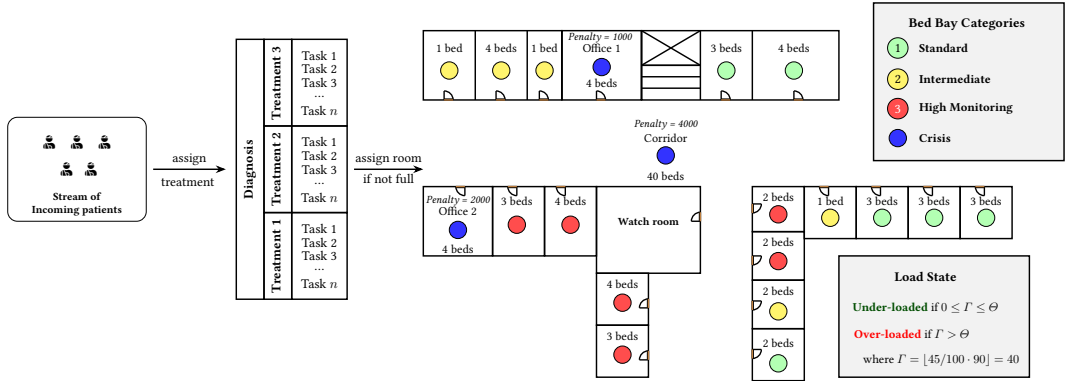
April 13, 2026



Contents

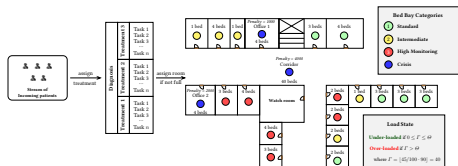
- ① Rationale
- ② Digital Twin Concepts
- ③ Digital Twin Architecture

Why do we need it?



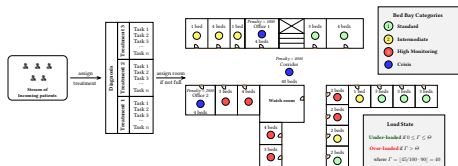
Why do we need it?

- Bed allocation is complex



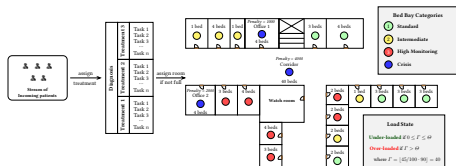
Why do we need it?

- Bed allocation is complex
- Admission is often handled manually



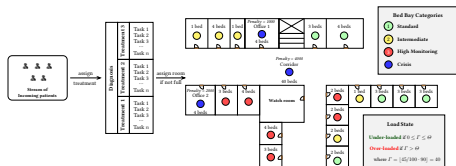
Why do we need it?

- Bed allocation is complex
- Admission is often handled manually
- Depends on several factors



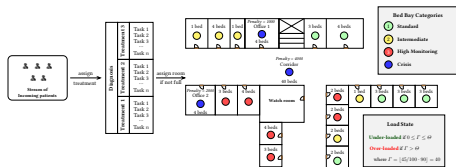
Why do we need it?

- Bed allocation is complex
- Admission is often handled manually
- Depends on several factors
- Different rooms have different monitoring categories
- With different simulations we can prepare for what-if scenarios



Why do we need it?

- Bed allocation is complex
- Admission is often handled manually
- Depends on several factors
- Different rooms have different monitoring categories
- With different simulations we can prepare for what-if scenarios
- During crisis wards might require more rooms (e.g. corridors, free offices)



What is the goal

- Allocate patients into rooms

What is the goal

- Allocate patients into rooms
- Save intermediate steps

What is the goal

- Allocate patients into rooms
- Save intermediate steps
- Minimise changes for patients already allocated

What is the goal

- Allocate patients into rooms
- Save intermediate steps
- Minimise changes for patients already allocated
- Ensure the constraints are met

What is the goal

- Allocate patients into rooms
- Save intermediate steps
- Minimise changes for patients already allocated
- Ensure the constraints are met
- Adjust resources in terms of rooms as required

Contents

- ① Rationale
- ② Digital Twin Concepts
 - Modeling aspects
- ③ Digital Twin Architecture

What is a Digital Twin

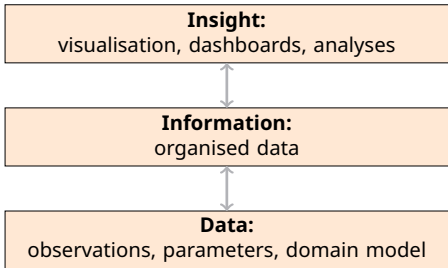
Digital twins are, as defined by NASEM

NASEM Definition of DT (2024)

A digital twin is a set of virtual information constructs that mimics the structure, context, and behaviour of a natural, engineered, or **social system** (or system-of-systems), is dynamically updated with data from its physical twin, has a predictive capability, and **informs decisions** that realise value. The bidirectional interaction between the virtual and the physical is central to the digital twin^a.

^aNational Academies of Sciences, Engineering, and Medicine (NASEM) (2024): Foundational Research Gaps and Future Directions for Digital Twins. The National Academies Press

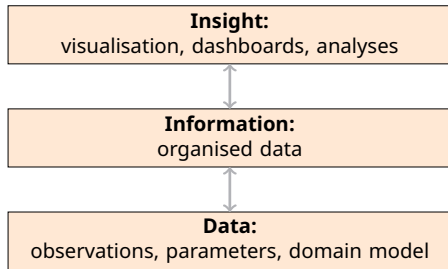
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")

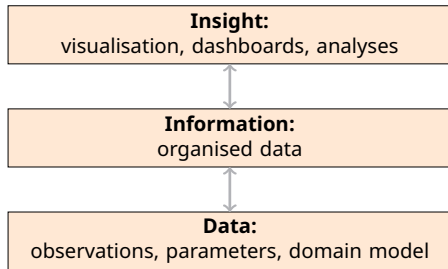
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")
- **Predictive:** Understand the future ("what may happen")

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")
- **Predictive:** Understand the future ("what may happen")
- **Prescriptive:** Advise on possible outcomes ("what if / what should")

Digital Twins components

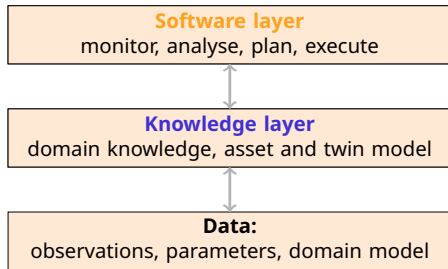
Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")
- **Predictive:** Understand the future ("what may happen")
- **Prescriptive:** Advise on possible outcomes ("what if")

What next:

- **Reactive:** Automated decision making

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")
- **Predictive:** Understand the future ("what may happen")
- **Prescriptive:** Advise on possible outcomes ("what if")

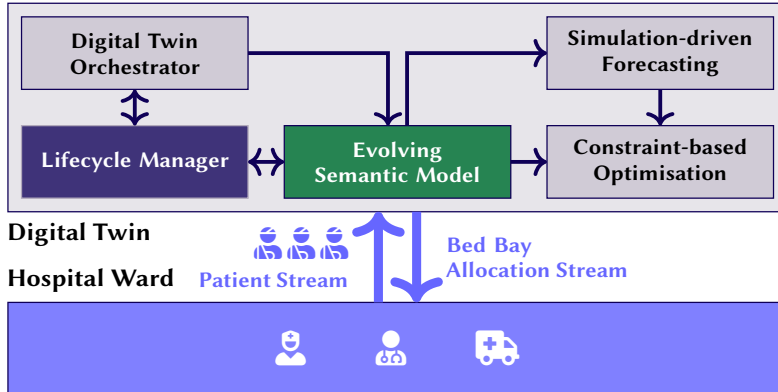
What next:

- **Reactive:** Automated decision making

Contents

- 1 Rationale
- 2 Digital Twin Concepts
- 3 Digital Twin Architecture**
 - Bedreflyt DT Execution

Architecture

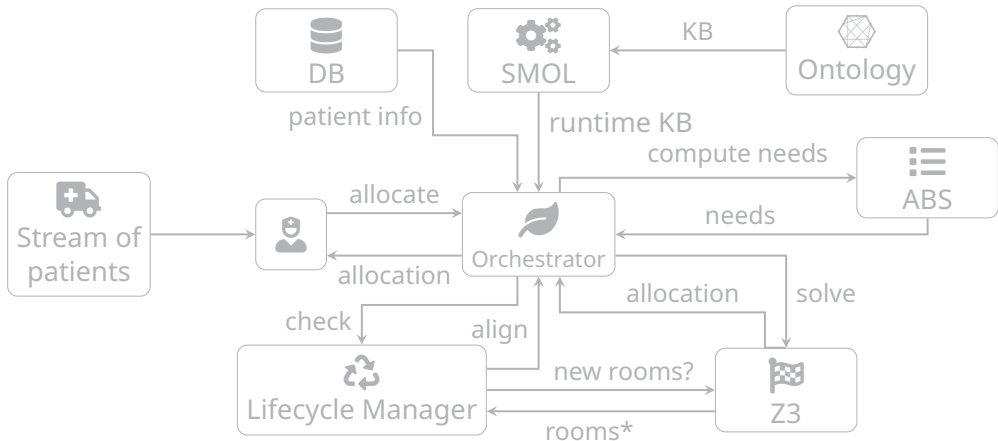


Digital Twin
Hospital Ward

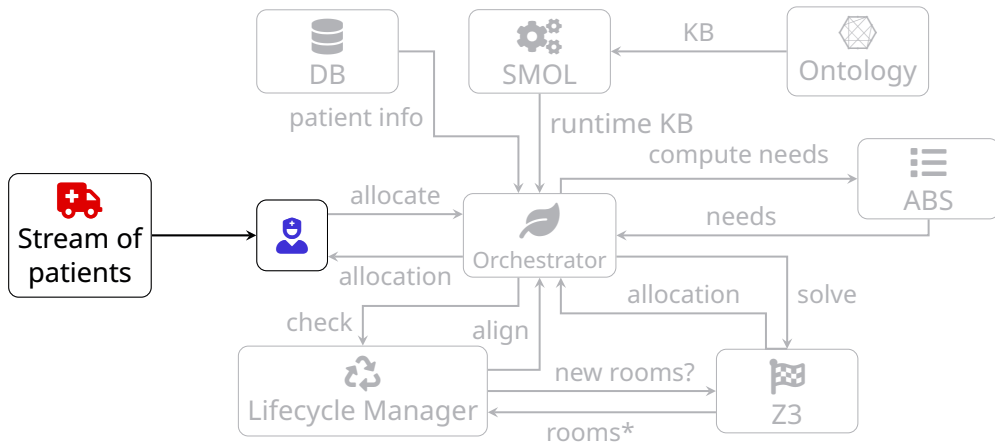
Patient Stream

Bed Bay
Allocation Stream

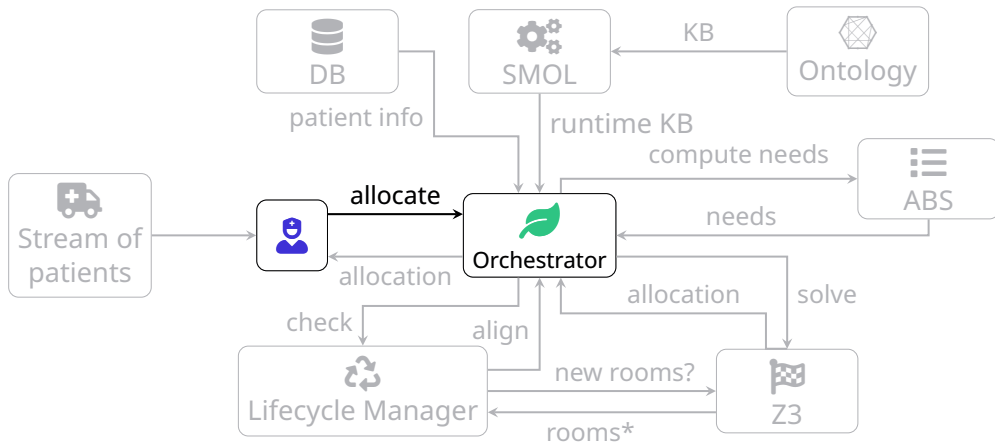
Allocation of patients



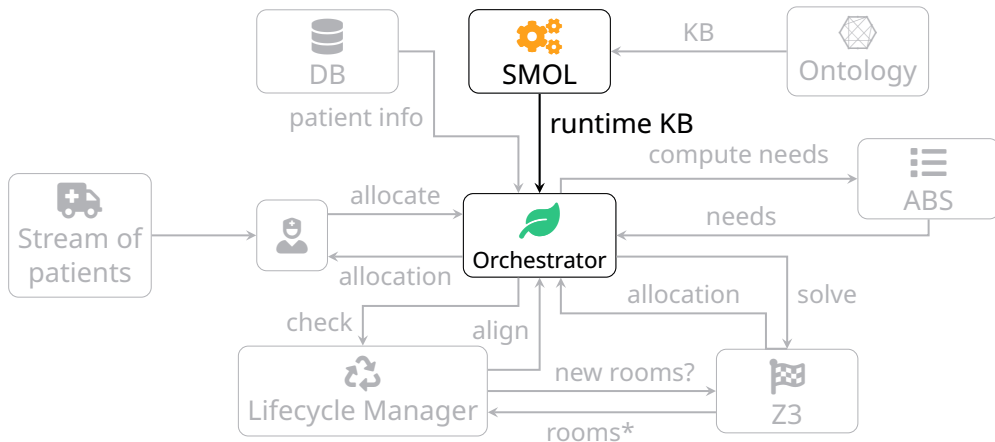
Allocation of patients



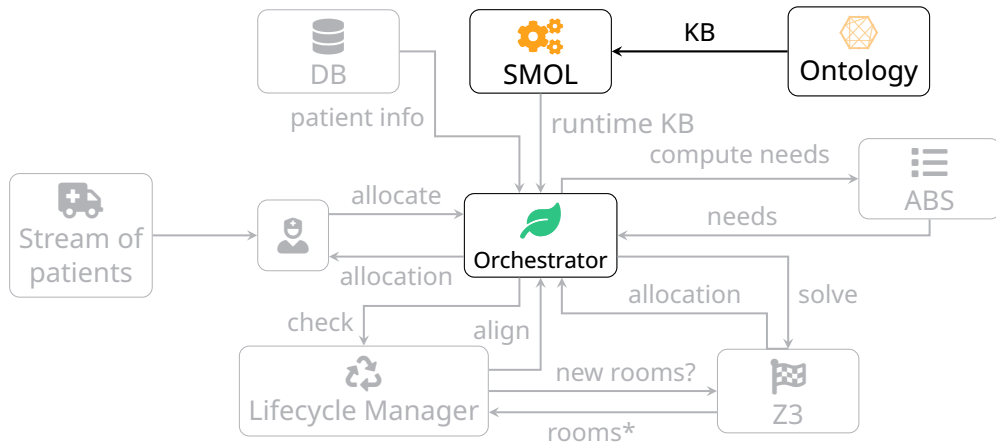
Allocation of patients



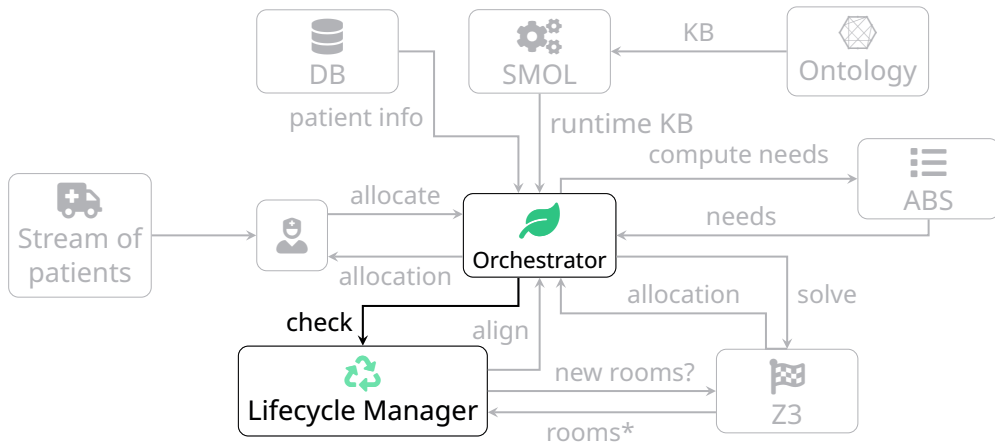
Allocation of patients



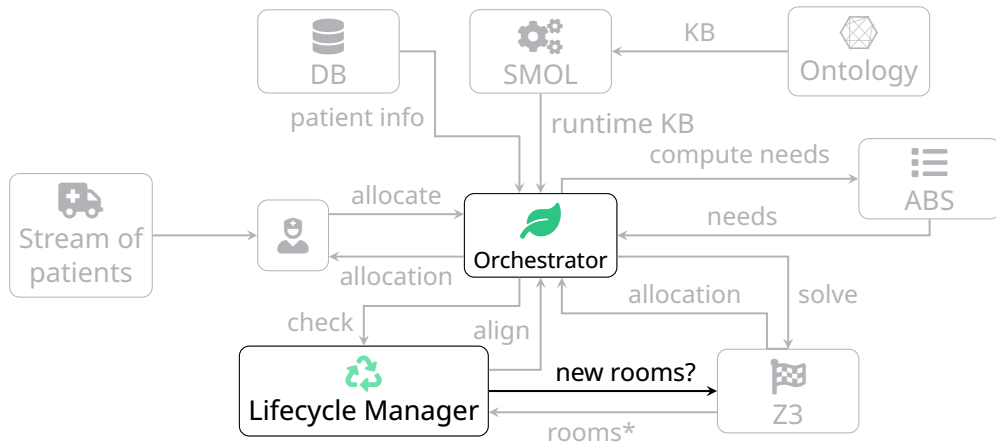
Allocation of patients



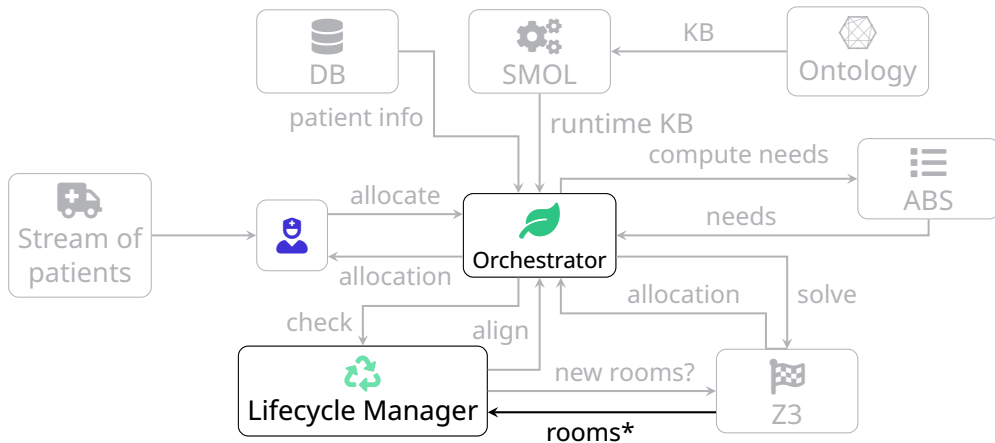
Allocation of patients



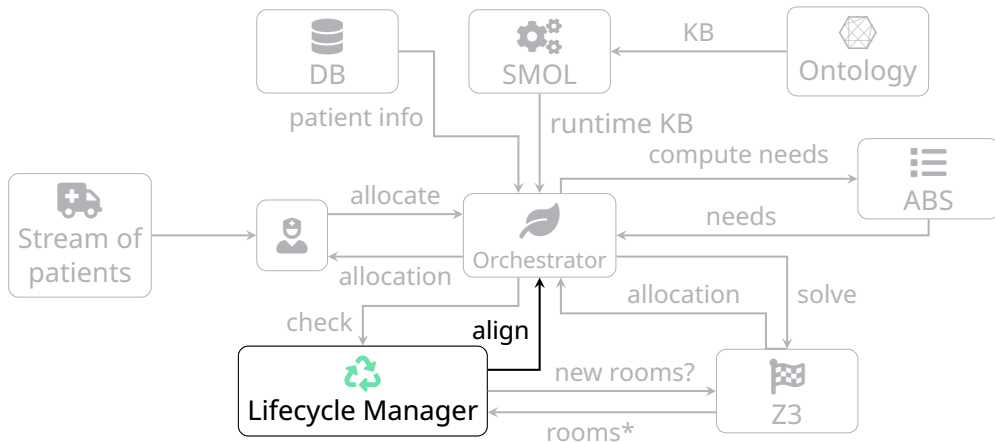
Allocation of patients



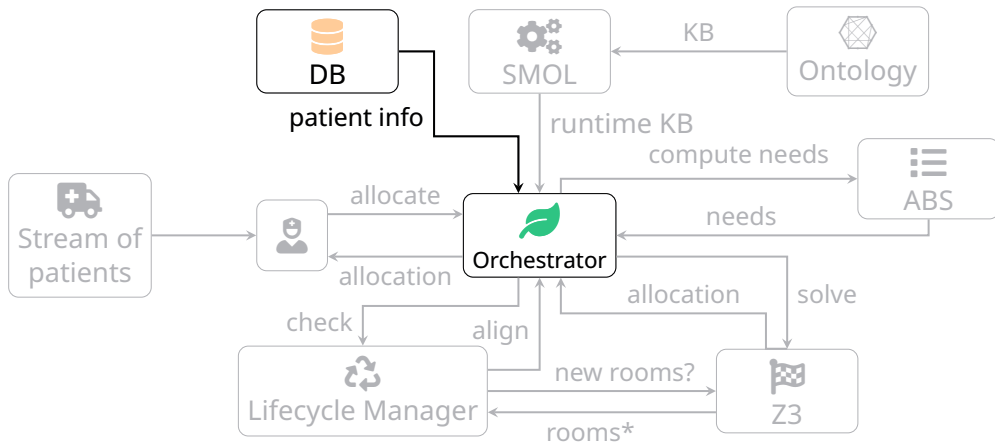
Allocation of patients



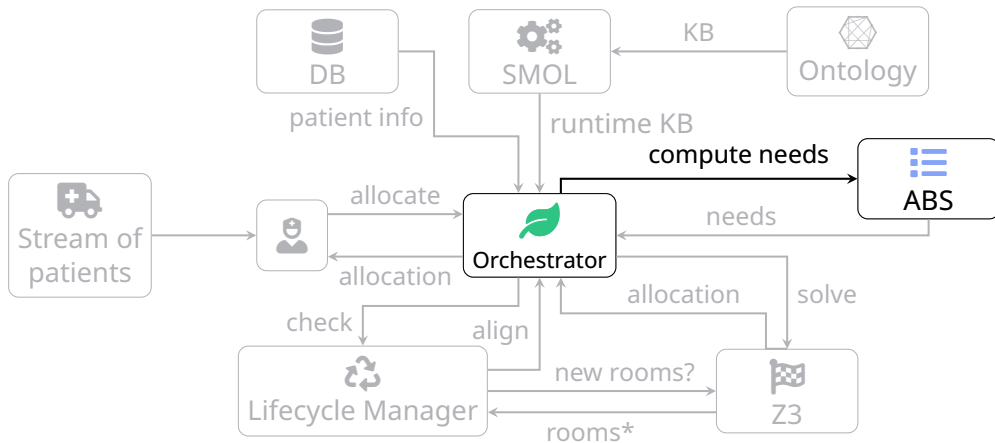
Allocation of patients



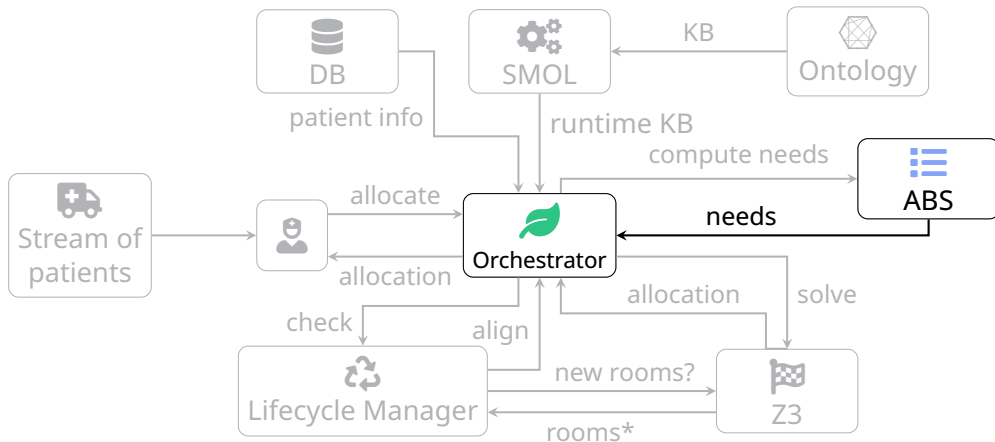
Allocation of patients



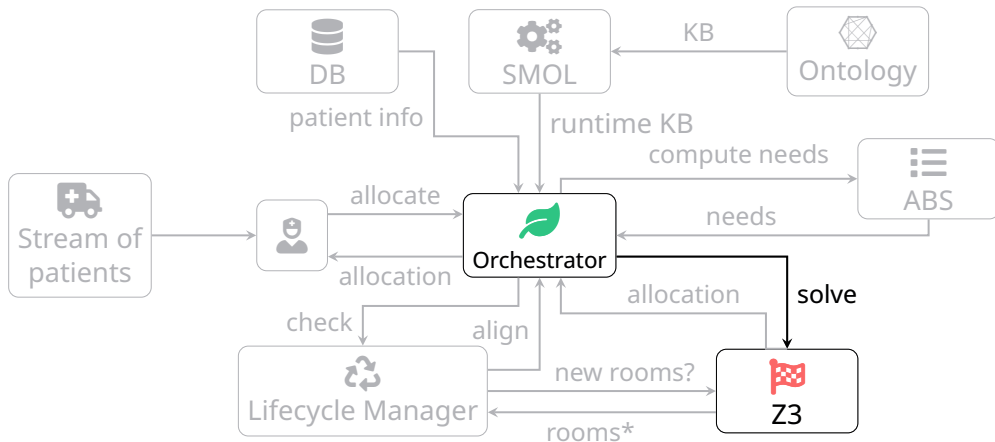
Allocation of patients



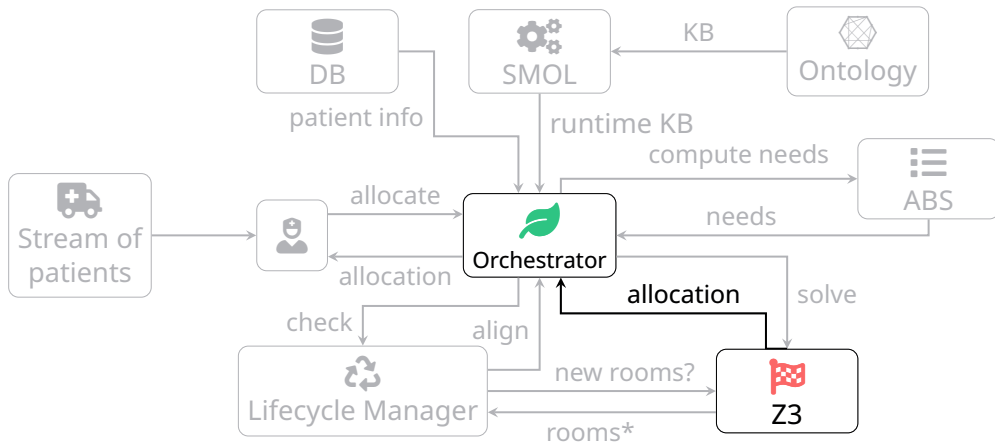
Allocation of patients



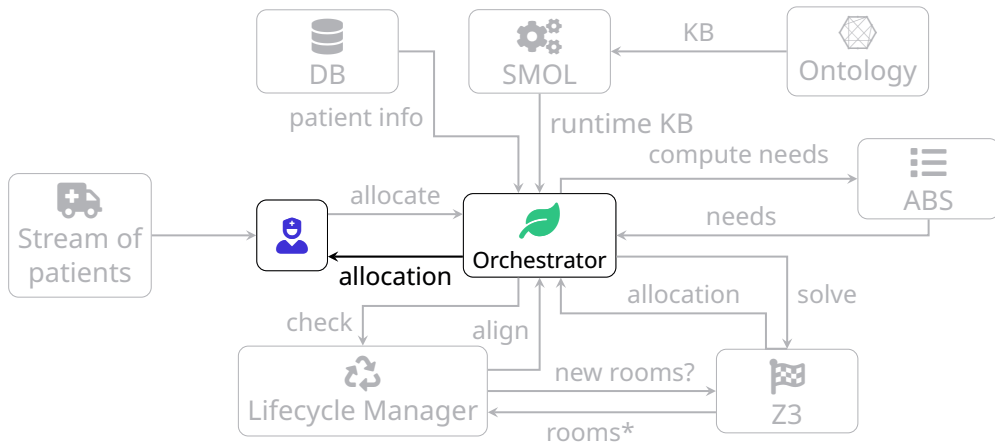
Allocation of patients



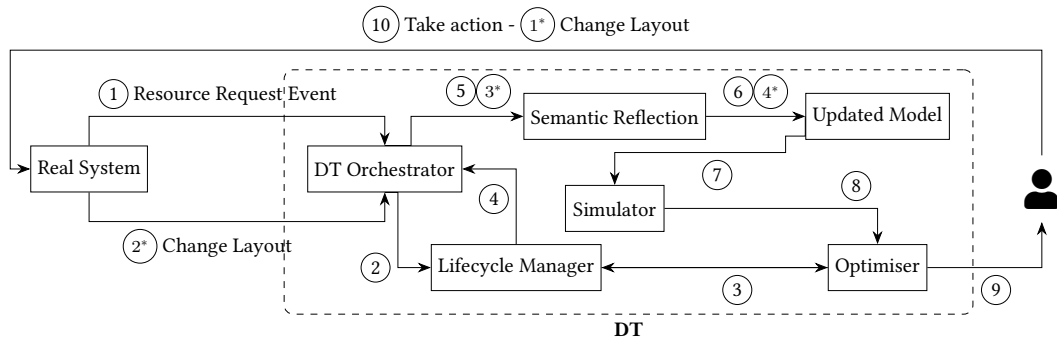
Allocation of patients



Allocation of patients



Human-in-the-loop



Human-in-the-loop (cont.)

- The human-in-the-loop is the one that will ultimately take action

Human-in-the-loop (cont.)

- The human-in-the-loop is the one that will ultimately take action
- Being it the suggestion for allocation, or to change layout

Human-in-the-loop (cont.)

- The human-in-the-loop is the one that will ultimately take action
- Being it the suggestion for allocation, or to change layout
- Depending on the choice, the system will adapt the model accordingly

Human-in-the-loop (cont.)

- The human-in-the-loop is the one that will ultimately take action
- Being it the suggestion for allocation, or to change layout
- Depending on the choice, the system will adapt the model accordingly
- If the layout changes, semantic reflection will ensure alignment to the changes

Technical components

- The orchestrator and the lifecycle manager are both implemented through a **Backend API** to handle the requests

Technical components

- The orchestrator and the lifecycle manager are both implemented through a **Backend API** to handle the requests
- The dynamic information for the allocation are saved in a **PostgreSQL** database

Technical components

- The orchestrator and the lifecycle manager are both implemented through a **Backend API** to handle the requests
- The dynamic information for the allocation are saved in a **PostgreSQL** database
- The data of the treatments and hospital structure is retrieved by **SMOL** to create a runtime Knowledge-Base

Technical components

- The orchestrator and the lifecycle manager are both implemented through a **Backend API** to handle the requests
- The dynamic information for the allocation are saved in a **PostgreSQL** database
- The data of the treatments and hospital structure is retrieved by **SMOL** to create a runtime Knowledge-Base
- The patient needs are computed by **ABS** (an actor-based programming language)

Technical components

- The orchestrator and the lifecycle manager are both implemented through a **Backend API** to handle the requests
- The dynamic information for the allocation are saved in a **PostgreSQL** database
- The data of the treatments and hospital structure is retrieved by **SMOL** to create a runtime Knowledge-Base
- The patient needs are computed by **ABS** (an actor-based programming language)
- We solve the allocation by solving the constraints with **Z3**

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer
smolang.org



Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

SMOL: Semantic Model Object Language

- Smalll OO programming system
- Ontology reasoners allow querying the KB
- Used to create the digital model

smolang.org



Twinning in SMOL

Programming support for twins with a behavioural and a structural layer
smolang.org

behavioural twins in SMOL

- SMOL can encapsulate (simulation) models based on the FMI standard
- Can automatically adapt the object states in the KB at runtime through semantic lifting



Simulation Environment

- To test the allocation process with adaptation we further added an Event Stream Generator (ESG)

Simulation Environment

- To test the allocation process with adaptation we further added an Event Stream Generator (ESG)
- ESG consists of an *Event Timestamp Generator* (ETG) and an *Event Sample Space* (ESS).

Simulation Environment

- To test the allocation process with adaptation we further added an Event Stream Generator (ESG)
- ESG consists of an *Event Timestamp Generator* (ETG) and an *Event Sample Space* (ESS).
- The ETG component generates the timestamp of the event, and the ESS component generates the event instance from a given set of possible event instances.

Simulation Environment

- To test the allocation process with adaptation we further added an Event Stream Generator (ESG)
- ESG consists of an *Event Timestamp Generator* (ETG) and an *Event Sample Space* (ESS).
- The ETG component generates the timestamp of the event, and the ESS component generates the event instance from a given set of possible event instances.
- For every sample, a constraint check is made, and if it does not satisfy the scenario-defined constraint, a resample is triggered.

Simulation Environment

- To test the allocation process with adaptation we further added an Event Stream Generator (ESG)
- ESG consists of an *Event Timestamp Generator* (ETG) and an *Event Sample Space* (ESS).
- The ETG component generates the timestamp of the event, and the ESS component generates the event instance from a given set of possible event instances.
- For every sample, a constraint check is made, and if it does not satisfy the scenario-defined constraint, a resample is triggered.
- We allow to account for patient peaks over time, to mimic a period of time of high stress in the ward, switching between high load and low load

Future directions

- Extend the work to enable self-adaptive techniques for different wards

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Explore different components like MDPs or Gurobi for optimisation

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Explore different components like MDPs or Gurobi for optimisation
- Include different components to simulate (e.g. staff management)

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Explore different components like MDPs or Gurobi for optimisation
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Explore different components like MDPs or Gurobi for optimisation
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital
- Explore different strategies based on uncertainties of non-adaptive approach.

Future directions

- Extend the work to enable self-adaptive techniques for different wards
- Subdivide the orchestrator into smaller units to allow for more modular infrastructure
- Explore different components like MDPs or Gurobi for optimisation
- Include different components to simulate (e.g. staff management)
- Set up a live stream of patients from the hospital
- Explore different strategies based on uncertainties of non-adaptive approach.
- Currently, SMOL provides overhead in the execution, leading to a trade-off between correctness and performance. We plan on investigate and improve the overall performance.